IntentLink — PRD & Design

Oct 10, 2026 · @abhi

Overview

IntentLink is a Chrome extension that carries a user's prompt from an agent chat into a real website, then reshapes that page around the prompt: matches highlighted, clutter dimmed or hidden, and a direct answer pinned at the top. The page stays the real page, so the cart, checkout and links keep working.

Problem. Big retail sites (Amazon, Walmart) are built for browsing and ads, not for answering a specific question. Agentic browsers answer in text and take you away from the page. Nothing hands you back a page that is already shaped around what you asked.

Concept: intent handoff. The user states their intent in a chat they already use, such as ChatGPT or Claude. The chat replies with an IntentLink: the site's search URL plus the prompt in the fragment, `#ll-intent=<URL-encoded prompt>`. Clicking it opens Chrome, and the extension reshapes the page. The fragment never reaches the site's server. Because any chat can write the link, IntentLink is a candidate open convention for agents as first-class visitors (AX).

Pitch. Personalization from stated intent, not guessed intent. "Any agent can make an IntentLink."

Context. Built in one day for The Living Internet Build Day at AGI House, presented by Coframe. Themes that this hits: generative UI, personalization engines, and interfaces built for agents. Demo runs on a shared laptop; nothing is deployed.

Goals and non-goals

The build succeeds if an IntentLink clicked in ChatGPT or Claude opens Amazon already reshaped in under 2 seconds, and a refine prompt typed on the page reshapes it live without a reload.

Goals

- In ChatGPT or Claude, a prompt yields a clickable IntentLink; clicking it opens Amazon search results already reshaped.
- A refine box on the reshaped page reshapes it again in place, with no reload.

- The intent travels only in the URL fragment; the extension has no UI outside the page.
- Every live dependency (site, network, model) has an offline fallback for the demo.

Non-goals

- No chat UI of our own and no side panel; the chat is whatever the user already uses.
- No deployment, backend server, accounts or Chrome Web Store listing.
- No fully generated replacement website; the real page is always underneath.
- No agent that clicks, buys or fills forms on the user's behalf.
- No support for sites beyond Amazon (and optionally Uniqlo, see the viability check), except the stretch generic fallback.
- No framework or build step: vanilla JS only.

Use cases and user flows

Three use cases are core and must work; one is stretch.

#	Use case	Priority	What the user does	What the user sees
UC1	Chat → reshaped search results (Amazon)	Core	Types "noise-cancelling headphones under $100, good for flights" in ChatGPT or Claude, then clicks the link	Results with matches highlighted and "why it fits" chips; non-matches dimmed; sponsored items hidden; summary bar: "4 of 48 match"
UC2	Chat → product page, answer first	Core	Pastes a product link plus "will this fit in a carry-on, and do people complain about the hinge?", then clicks the link	Answer card at top; relevant spec rows and 2–3 review snippets highlighted; rest collapsed
UC3	Refine on the page	Core	Types "only over-ear" in the refine box on the summary bar	Page reshapes live, no reload

#	Use case	Priority	What the user does	What the user sees
UC4	Unknown site (Best Buy or Target)	Stretch	Clicks an IntentLink to a new domain	Model writes selectors once, they are cached, and the second load reshapes in under 1 second

Main flow (UC1)

1. In ChatGPT or Claude (a Custom GPT or Project with the IntentLink instructions), the user types their intent.
2. The chat replies with a markdown link: the site's search URL plus `#ll-intent=<URL-encoded prompt>`.
3. The user clicks the link, and Chrome opens the page.
4. The content script reads the fragment and extracts about 30 compact result items.
5. The worker's rank call returns which items to keep and why; JS re-checks numeric constraints.
6. The content script applies CSS classes and injects the summary bar with a refine box.

Refine flow. The user types "only over-ear" in the refine box. The rank call runs again on the already-extracted items with the combined intent, and the page re-applies classes with no reload. Asking the chat for a new link also works.

UC2 flow. The user pastes a product URL and a question into the chat, which returns that URL with the question as the fragment. The content script extracts title, bullets, spec table rows and the top ~30 reviews, each tagged with an id. One model call returns an answer plus the ids of supporting elements.

Optional: clothing site (Uniqlo). A second site in a different domain would show that IntentLink handles fit, fabric, occasion and price, not only specs, for example "linen shirt for a beach wedding, under $50." It is an explore-only option, not a requirement. Do the viability check first; if any check fails, skip it and fall back to Walmart or no second site.

Viability check (about 10 minutes, in DevTools on a Uniqlo search page):

- [ ] Search results put the query in the URL, so the chat can build the link
- [ ] Each result's title, price, color and sale tag are in the DOM or an embedded JSON blob, readable without scrolling or hovering
- [ ] Material or fit words appear in titles or listing text; the model ranks on text only, not images

- [ ] Size and stock appear on the listing page. If not, drop "in my size" and rank on fabric, fit, price and occasion
- [ ] No bot check or login wall in a signed-in clean profile, and results load without infinite scroll

Zara is out of scope: its layout is image-first with little text per item and it is likely to be a time sink.

Functional requirements

P0 is the demo; P1 makes it impressive; P2 only after P0 and P1 are solid and the backups exist.

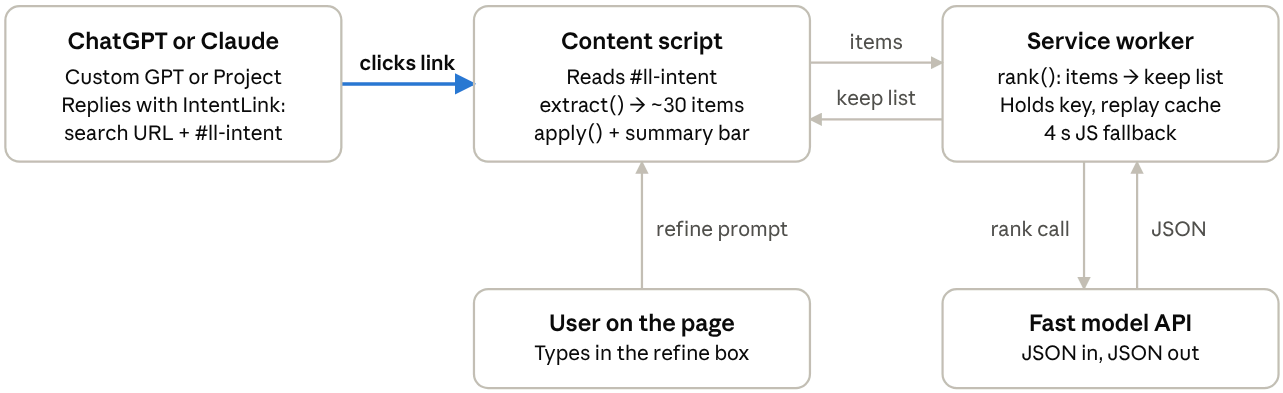
ID	Requirement	Priority
F1	Chat-side link generator: Custom GPT and Claude Project instructions that turn a prompt into `searchUrl#ll-intent=<URL-encoded prompt>`, returned as one clickable markdown link	P0
F2	Content script reads and decodes `#ll-intent` on load (tolerant of `+` and bad encoding); does nothing on pages without it	P0
F3	Worker captures `#ll-intent` per tab via `chrome.webNavigation` in case a redirect drops the fragment	P0
F4	Amazon search adapter extracts `{id, title, price, rating, reviewCount, prime, sponsored}` per result and tags each node with `data-ll-id`	P0
F5	Rank: `{intent, items}` → `{filters, summary, keep:[{id, reason}]}` via one call in the worker, behind a RANK_ENGINE switch ("llm" or "jev")	P0
F6	Deterministic re-check of numeric/boolean `filters` (price, rating, Prime) after F5	P0

ID	Requirement	Priority
F7	Apply in two passes: pass 1 from the adapter and JS filters at extraction; pass 2 from rank. Toggle `ll-keep`, `ll-dim`, `ll-hide`; hide sponsored items and carousels; inject a pinned summary bar; render reason chips on kept items	P0
F8	Refine box in the summary bar re-runs F5–F7 with the combined intent, no reload	P0
F9	Reset control in the summary bar removes all IntentLink classes and the bar	P0
F10	Latency badge on the summary bar (click-to-reshape time in ms)	P1
F11	Product page adapter + answer card (UC2)	P1
F12	"Copy IntentLink" in the summary bar, sharing the current refined intent	P1
F13	Optional second site: Uniqlo adapter if the viability check passes (see Use cases), or Walmart via the `__NEXT_DATA__` JSON blob	P2
F14	Generic fallback: model writes selectors for an unknown domain once; cache them in `chrome.storage` (UC4)	P2
F15	Local MCP tool `make_intentlink` so Claude Desktop builds links deterministically	P2

Architecture

Any chat writes the link; inside Chrome only the service worker calls a model, and the page only toggles CSS classes, so a failure never breaks the page.

Any chat writes the link; only the extension calls the model

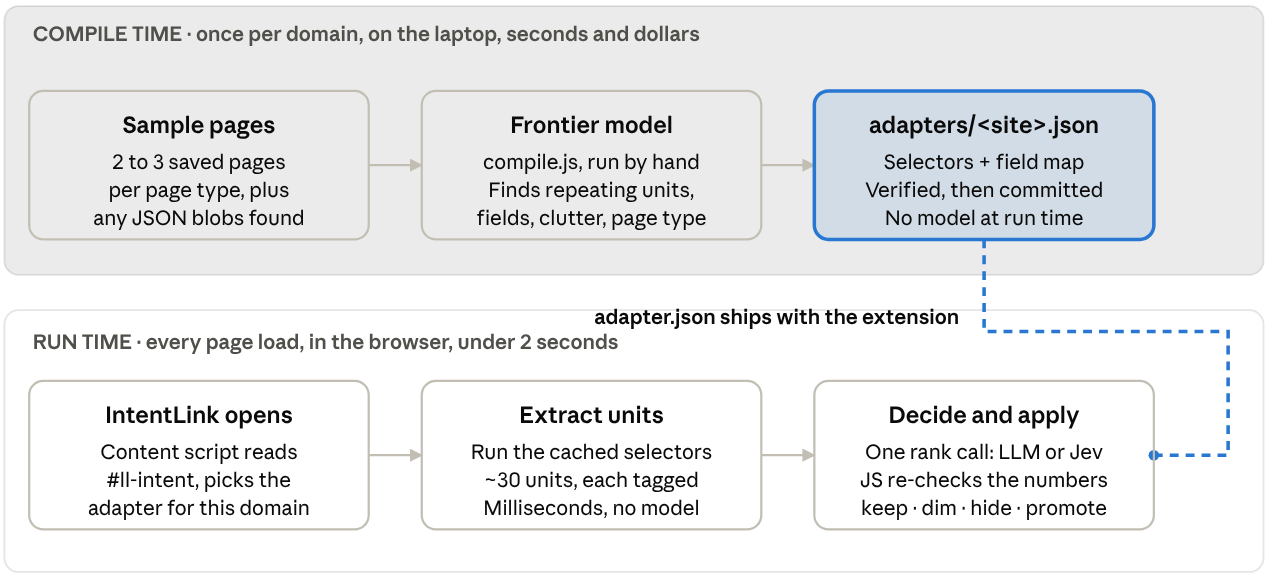


IntentLink architecture · chat → page → worker → model

The highlighted arrow is the handoff: the only thing that crosses from the chat into Chrome is the link.

Two tiers: compile time and run time. This split is what makes IntentLink generalize past the sites we hand-write. A frontier model compiles a site adapter once, offline; every page load then runs that cached adapter plus one small decision call.

A frontier model compiles each site once; every page load runs the cached adapter



Two tiers · compile a site once, run the adapter on every load

	Compile time	Run time
How often	Once per domain, re-run when the site changes	Every page load and every refine
Where	`compile.js` on the laptop, run by hand	Content script and service worker
Model	A frontier model; 30 s and a few dollars is fine	A small fast model or Jev, or none at all
Output	`adapters/<site>.json`, committed to the repo	A reshaped page
If it fails	We notice before the demo and fix the JSON by hand	Timeout fallback to keyword filtering

Pitch line: *a frontier model compiles a site once, and after that every visitor gets a page shaped around their intent in a few hundred milliseconds.* Adding a site costs a compile run, not an afternoon of hand-written selectors.

What the compiler produces. `compile.js` takes a URL or a saved page, strips scripts and styles, and sends a skeleton of the DOM (tag, classes, `data-*` attributes, text length, repeat count per subtree) plus any structured data it finds. It is given no site-specific knowledge, so the same script runs on any site. It returns, and we commit:

```
{ "domain": "amazon.com",
  "pageTypes": [ { "name": "search", "match": "/s?",
      "unit": "[data-component-type='s-search-result']",
      "unitId": "data-asin",
      "fields": { "title": "h2 span", "price": ".a-price .a-offscreen",
                  "rating": "[aria-label$='out of 5 stars']", "prime": "
[aria-label*='Prime']" },
      "clutter": [ "[data-component-type='sp-sponsored-result']", ".a-
carousel-container" ],
      "structured": null } ],
  "compiledAt": "2026-10-10", "verifiedOn": ["headphones", "running shoes",
"desk lamp"] }
```

How the compiler finds candidates, with no site-specific rules

1. Prefer structured data when present: JSON-LD `Product` nodes, `__NEXT_DATA__`, or an XHR response carrying the listing array. Then no selectors are needed for those fields.

2. Otherwise find the repeating unit: the deepest subtree repeated 8 or more times with a stable class or `data-*` signature and similar inner text length.
3. Within one unit, label the fields by asking the model what each text run is (title, price, rating, badge), using a few real examples rather than guessing from class names.
4. Mark clutter: repeated blocks outside the unit list, and units whose signature includes sponsored or ad markers.
5. Self-verify: run the candidate selectors against a second sample page. If the unit count or field fill rate drops sharply, retry once with the failures fed back, then flag the domain as needing a hand fix.

Units and ops, the general model. The primitive is a *unit with a role*, not "an item in a list." On a search page the units are results; on a record page they are sections and spec rows. Every reshape is one of six ops: keep, dim, hide, promote (move to the top), collapse, answer (inject a card). The adapter says what the units are, and the rank call picks the op. That is how the same runtime covers a car inventory search, a vehicle detail page or a comparison table without new runtime code.

Scope boundary. IntentLink reshapes what a page *displays*. It does not drive stateful flows: a car configurator, a multi-step checkout or anything where the answer requires searching option combinations is out of scope, and so is anything needing a click. For the demo we compile only list-shaped retail sites: Amazon, Walmart and Uniqlo.

Two-pass reshaping. The page never waits on a model. Pass 1 runs the moment the units are extracted, using only the cached adapter and plain JS; pass 2 refines it when the rank call returns. Both passes write the same classes, so pass 2 is a diff, not a redraw.

	Pass 1: instant	Pass 2: refined
When	At extraction, typically under 200 ms	When rank returns, 0.3–2 s
Decided by	Cached adapter + JS: hide `clutter` selectors, apply numeric and boolean filters parsed from the intent by regex (price, rating, Prime), keyword match on titles	LLM or Jev over the extracted units
What it writes	`ll-hide` on clutter, `ll-dim` on numeric misses, `ll-pending` on everything still undecided	`ll-keep` with reason chips, `ll-dim` on the rest, `promote` on the top matches

	Pass 1: instant	Pass 2: refined
Summary bar	"12 of 48 match the filters · refining…" with a spinner	"4 of 48 match: under $100, 4★+, Prime" plus the latency badge

Rules that keep this honest:

- Pass 1 only ever hides what the adapter marked as clutter and dims what provably fails a numeric or boolean constraint. It never hides a unit on a guess, so pass 2 can only narrow the set, never resurrect something the user saw vanish.
- `ll-pending` is a neutral state, not a dim: undecided units stay fully readable, so a slow or failed pass 2 leaves a usable page.
- Transitions are a 150 ms opacity fade, so the refinement reads as the page settling rather than flickering.
- If pass 2 fails or times out, pass 1's result stays and the bar says "filtered locally".
- With Jev, its per-item probability drives the strength: above 0.8 is a full highlight, 0.5 to 0.8 gets a lighter "maybe" ring, below 0.5 is dimmed. With the LLM engine, every kept item is simply a full highlight.

This is also the demo's best visual: the page visibly settles from 12 candidates to 4 in front of the judges, which makes the compile/run split legible without explaining it.

Components

- **Chat side (no code):** a Custom GPT and a Claude Project with instructions that turn the user's intent into an IntentLink. Stretch: a local MCP tool `make_intentlink` for deterministic links in Claude Desktop.
- **`background.js` (service worker):** `rank()` behind the RANK_ENGINE switch, response cache, 4 s timeout fallback; holds the API keys; captures `#ll-intent` per tab via `chrome.webNavigation`.
- **`content.js` + `adapters/`:** reads the intent, `extract()`, `apply(plan)`, `reset()`, and renders the summary bar with the refine box. Matches `amazon.com/*` and `localhost/*`.
- **`reshape.css`:** `.ll-keep`, `.ll-dim { opacity: .15 }`, `.ll-hide { display: none }`, summary bar, reason chips.

IntentLink format. The site URL built by the chat (using the site's own search params where possible) plus the prompt as plain URL-encoded text. Plain text replaces base64 JSON because chat models produce it reliably.

```
https://www.amazon.com/s?k=noise+cancelling+headphones#ll-intent=noise-cancelling%20headphones%20under%20%24100%2C%20good%20for%20flights
```

Chat instructions (draft for the Custom GPT and Claude Project)

```
You turn shopping requests into IntentLinks.
1. Pick the site (default amazon.com). Build its search URL from the key product words:
   https://www.amazon.com/s?k=<words joined by +>
   If the user pasted a product URL, use that URL instead.
2. Append #ll-intent= followed by the user's full request, URL-encoded.
3. Reply with one markdown link titled "Open on Amazon, shaped for you" and nothing else.
```

Rank contract. Both engines share this contract. Worker → model input: `{intent, items:[{id, title, price, rating, reviewCount, prime}]}`. Output:

```
{ "filters": { "priceMax": 100, "ratingMin": 4.0, "prime": true },
  "summary": "4 of 48 match: under $100, 4★+, Prime",
  "keep": [ { "id": "ll-7", "reason": "ANC, 30h battery, $79" } ] }
```

Ids not in `keep` are dimmed and sponsored items are hidden. JS then drops any kept item that breaks `filters`.

Messages. Content script → worker: `{type: "rank", intent, items}`. A refine reuses it with `intent = original + " / " + refinement`. The worker replies with the plan.

Optional rank engine: Jev. Jev (TypeSafe's first System One model) takes unstructured state plus predefined questions and returns typed answers with calibrated probabilities. It cannot generate free text. Its published figures are 70–500 ms per call and $0.042 per million input tokens with free output (announcement); those are the vendor's own numbers on their benchmarks, so time our real calls before quoting any of them.

Why it suits `rank()`: it is a fuzzy decision over a list, which is exactly its shape; it cannot return malformed output, which removes the invalid-JSON risk; and per-item probabilities let the page show a strong highlight for confident matches and a faint "maybe" ring for borderline ones.

It stays optional and must be switchable in one place:

- `RANK_ENGINE` in `config.local.js` is `"llm"` by default. Setting it to `"jev"` is the only change needed; no other file knows which engine ran.

- Both engines implement the same `rank(intent, items)` contract above, so `apply()` is unchanged.
- Any Jev error or timeout falls back to the LLM engine for that call and logs it. The demo never depends on Jev being up.
- Jev cannot write prose, so the summary line and "why it fits" chips are assembled in JS from its per-question answers ("ANC ✓ · $79 · 4.6★"). The UC2 answer card stays on the LLM.
- Per item, ask a few typed questions: matches the stated intent; is sponsored or an ad; meets the price and rating limits; has the specific feature named in the prompt.

Go/no-go spike, 30 minutes, only after the P0 path works:

- [ ] Early access granted and a key in hand (apply at the start of the day)
- [ ] 30 real Amazon items with 4 questions return in under 500 ms end to end
- [ ] Keep/dim decisions look right on 3 rehearsed prompts, against the LLM engine's output
- [ ] Item count fits the API's limits (the post mentions a cardinality limit of 255)

If any check fails, leave `RANK_ENGINE` on `"llm"` and mention Jev as an experiment, not as a claim.

Non-functional requirements

The demo must never show a blank or broken page, even with no network.

Latency

- Target: under 2 s from page load to reshape for UC1, under 1 s for follow-ups.
- Never send raw HTML to a model. Ranking input is at most ~30 items of about 150 characters each.
- Use a small, fast model with JSON output, or Jev when enabled. One call per page load (rank), since the chat already built the search URL; one per refine.
- Start extraction at page load and paint pass 1 before the ranking call returns (see Two-pass reshaping).

Reliability and fallbacks

- Reshaping only toggles classes and adds elements. The page's own HTML is never rewritten, so a failure leaves the normal page.
- Timeout: if a model call takes over 4 s, fall back to keyword-and-price filtering in JS.

- Replay cache: responses are cached by `hash(prompt + page URL)`. A "demo mode" toggle serves cached answers with no network.
- Snapshot fallback: demo pages saved as "Webpage, Complete" and served by `python -m http.server`. The content script also matches `localhost`.
- Sponsored and carousel selectors fail gracefully when not found.

Privacy and security

- The intent lives in the URL fragment, which browsers do not send to the site's server.
- The API key lives in a local, git-ignored config file. The extension is loaded unpacked only.
- The extension never clicks, submits or purchases anything.

Environment

- Chrome, Manifest V3, loaded unpacked. A clean Chrome profile with only this extension, signed into Amazon to reduce bot checks.

Build plan

UC1 must work end to end by milestone 3; everything after it is additive. Hours are counted from the start of coding.

1. **Skeleton (hour 1).** Manifest, content script and service worker loaded unpacked. Done when a hand-written IntentLink logs the decoded intent in the page console.
2. **Extract and apply (hours 1–2).** Amazon search adapter and reshape CSS, using a hardcoded plan. Done when a fake plan visibly dims and highlights a real results page.
3. **Model loop (hours 2–3).** Rank call in the worker behind the RANK_ENGINE switch, with the LLM engine first; deterministic re-check; summary bar. Done when a hand-written IntentLink opens a reshaped page.
4. **Chat side (hours 3–4).** Write the Custom GPT and Claude Project instructions; test that both reliably emit clickable, correctly encoded links. *UC1 complete.*
5. **Refine and reset (hour 4).** Refine box and reset control in the summary bar (UC3).
6. **Safety net (hour 5).** Replay cache, demo mode, 4 s timeout fallback, saved snapshots on localhost. Record a backup video.
7. **Product answer card (hours 5–6).** UC2.
8. **Polish (hour 7).** Latency badge and Copy IntentLink (F10, F12).

9. **Stretch (hour 7+).** Jev spike behind the switch (see Architecture), optional second site (Uniqlo if the viability check passes, else Walmart), generic selector fallback, MCP tool.
10. **Freeze (last 45 min).** No new code; rehearse the demo.

Cut list, in order: MCP tool → generic fallback → Jev → second site (Uniqlo or Walmart) → reason chips → UC2. Never cut the refine box or the safety net.

Suggested file layout

```
intentlink/
  manifest.json
  background.js       # rank() switch, cache, timeout fallback, webNavigation
intent capture
  engines/llm.js      # default rank engine
  engines/jev.js      # optional rank engine, same contract
  content.js          # read intent, extract(), apply(), refine box, reset()
  adapters/amazon.js  # search + product extractors
  reshape.css
  config.local.js     # RANK_ENGINE flag + API keys, git-ignored
  chat/
    gpt-instructions.md     # Custom GPT instructions
    claude-project.md       # Claude Project instructions
  snapshots/          # saved demo pages
```

Demo script and success criteria

The demo is 90 seconds on the shared laptop, with snapshot tabs open in the background in case the live site fails.

1. Show a cluttered Amazon results page: "this is the frozen web."
2. In ChatGPT, type the UC1 prompt. It replies with an IntentLink.
3. Click it. Amazon opens already reshaped, with the latency badge visible.
4. Type "only over-ear" in the refine box. The page changes live, with no reload.
5. Switch to Claude, paste a product link and the UC2 question, and click its link. The answer card appears: "two different agents, one link format."
6. Close: "Today the extension renders it. Tomorrow sites read `ll-intent` themselves. Any agent can make an IntentLink."

Success criteria

- [ ] UC1 reshape under 2 s live, with a correct match count on 3 rehearsed prompts
- [ ] Follow-up reshape under 1 s with no reload
- [ ] No item that violates a numeric constraint is ever highlighted
- [ ] Demo verified end to end with RANK_ENGINE on "llm", whatever Jev does
- [ ] Full demo runs with Wi-Fi off (demo mode + snapshots)
- [ ] Backup video recorded

Risks and open questions

The biggest risk is Amazon's live page misbehaving during the demo; snapshots and demo mode cover it.

Risk	Likelihood	Mitigation
Chat model emits a malformed or unencoded link	Medium	Plain-text fragment, not base64 JSON; few-shot examples in the instructions; tolerant decoding (F2); MCP tool as stretch
Amazon CAPTCHA or bot check during demo	Medium	Signed-in clean profile; snapshot tabs on localhost
Amazon DOM differs from expected selectors (layout A/B tests)	Medium	Adapter checks several selectors per field; skip items missing a price; test on 3+ queries early
Venue Wi-Fi drops or the model API is slow	Medium	Demo mode replay cache; 4 s timeout with JS keyword fallback; chat links pre-generated and saved
A redirect drops the URL fragment	Low	Worker captures the intent from `chrome.webNavigation` before the redirect (F3)
Model returns invalid JSON or ignores constraints	Low	JSON-only prompt with schema; parse defensively; deterministic re-check (F6)
Too little time for UC2	Medium	UC2 is on the cut list after the stretch items

Jev carries its own risks, all contained by the switch: early access may not be granted in time, its real latency on our calls is unverified, and its question-based interface may not express the intent as well as a prompt does. In every case `RANK_ENGINE` stays on `"llm"` and nothing else changes.

Open questions

- [ ] Did anyone get Jev early access? Apply first thing; without a key the LLM engine is the default.
- [ ] Custom GPT and Claude Project both, or just one for the demo?
- [ ] Should reshaping hide non-matches entirely or only dim them by default? Dimming is safer for the demo story.
- [ ] Which three rehearsed prompts will the demo use?